CAPSTONE PROJECT
BRIJESH SINGH----------AI-MEETING-NOTES-ANALYZER

1. Introduction

Meetings often produce important decisions and follow-up work, but manually converting a long conversation into concise minutes is time-consuming. Participants may also leave a meeting without a consistent record of who owns each task or when it should be completed. This project explores how a language model, supported by a small workflow graph and local semantic retrieval, can assist with this post-meeting task. The application accepts plain-text notes or a speaker-labelled transcript. It analyzes the transcript in multiple stages and produces a report designed to make the meeting's main ideas and follow-up tasks easier to review.

1.1 Problem statement

Given a text transcript, produce useful, structured meeting notes while preserving the information stated by participants. In particular, the system should distinguish explicit commitments from general discussion, identify a responsible owner when one is stated, and communicate action urgency.

1.2 Objectives

To design and implement an AI-assisted meeting-notes analyzer using a LangGraph workflow and a Streamlit interface.

- Accept meeting transcripts as plain text, including speaker-labelled messages.
- Extract concise labels for the principal discussion topics.
- Generate a concise summary intended to contain three to five sentences.
- Identify explicit action items and their stated owners.
- Classify each detected task as High, Medium, or Low Priority.
- Skip the priority-classification node if there are no action items.
- Use local text embeddings to rank relevant transcript segments.
- Display the report in a browser-based Streamlit interface without exposing an API-key entry or visibility control.
- Keep the workflow schemas, validators, functions, and graph implementation in Jupyter notebook code cells without custom classes.

1.3 Scope

The implemented scope is transcript-in, structured-notes-out. Text is supplied by the user or preloaded from `Meeting_Transcript.txt`. The application does not record meetings, accept audio/video, perform speech recognition, identify speakers from voice, distribute minutes, or create tasks in external systems.

1.4 Application Summary

The AI Meeting Notes Analyzer is a Python application that transforms a user-provided meeting transcript into a structured report with exactly three sections: Meeting Summary, Key Topics, and Action Items. Each action item contains its task description, owner, and priority. If no action items are detected, the Action Items section displays `No action items identified in this meeting.` The application is implemented as a LangGraph workflow, presented through a Streamlit web interface, and developed in a Jupyter notebook. The language-model requests are sent through OpenRouter using the configured model identifier `nvidia/nemotron-3-ultra-550b-a55b:free`. Structured outputs are requested using JSON mode and validated by notebook-defined dictionary schemas and

validation functions. Transcript segments are embedded locally with `sentence-transformers/all-MiniLM-L6-v2`; cosine similarity selects relevant segments to include as additional evidence in each agent prompt. The full transcript is also included in those prompts. A conditional edge in the graph skips priority classification when the action-extraction step returns no tasks.

The application is a decision-support prototype, not a meeting recorder or source of authoritative minutes. It does not transcribe audio, store reports in a database, update task-management systems, or independently verify model claims. Its output depends on transcript quality, model behavior, API availability, and the configured service. Testing to date covers syntax, workflow paths with deterministic mocks, dependency consistency, and a Streamlit launch smoke test. A scored evaluation against a human-annotated dataset and live end-to-end model evaluation remain future work.

Keywords: LangGraph, meeting summarization, action-item extraction,sentence embeddings, OpenRouter, Streamlit, structured output

2. Key Deliverables

Deliverable	Location
Github Repository Link:	https://github.com/MyGit-App/MeetingScriptAnalyzer.git
Development notebook	MeetingTranscriptAnalyzer.ipynb
Model	`nvidia/nemotron-3-ultra-550b-a55b:free`
Streamlit Application	https://meetingtranscriptanalyzer.streamlit.app/
Dataset source	Meeting_Transcript.txt
Streamlit App File	app.py

3. Requirements and expected output

3.1 Functional requirements

ID	Requirement	Implementation
F-01	Accept a non-empty transcript	Streamlit text area and graph input node
F-02	Identify key topics	Structured topic-extraction node
F-03	Summarize the meeting	Structured summary node; function-based sentence-count validation
F-04	Extract explicit action items	Structured action-extraction node
F-05	Associate owners with actions	Owner field on each action; missing values normalized
F-06	Classify task priority	Conditional priority node returns one priority for each action
F-07	Skip classification when no tasks are found	Conditional graph edge routes directly to finalization
F-08	Produce a consolidated report	Finalization node constructs a report dictionary
F-09	Use local embeddings	CPU embeddings generated with Sentence Transformers
F-10	Hide API key from UI	Key read from environment, `.env`, or Streamlit secrets
F-11	Provide an interactive browser interface	Streamlit `app.py`
F-12	Keep workflow implementation in notebook code cells	`MeetingTranscriptAnalyzer.ipynb` contains dictionary schemas and functions

3.2 Final report format

The graph returns a `report` object with exactly these keys:

```
meeting_summary   Concise 3–5 sentence summary
key_topics        List of discussion-topic labels
action_items      List of tasks, owners, and priorities, or the no-action message
```

For a non-empty list, each action item has the fields `task_description`, `owner`, and `priority`. When the action list is empty,

`action_items` is the string `No action items identified in this meeting.`.

The Streamlit app renders exactly three report headings and does not display additional owner, overall-priority, or JSON sections.

4. System architecture

4.1 High-level architecture



4.2 Graph state

The workflow uses LangGraph's plain `dict` state schema. It carries the transcript, split segments, embedding ectors, and agent results through the graph. Each node returns the updated state dictionary together with its new fields so later nodes retain all previously computed values. The final report is placed in the `report` state field.

4.3 Agents and their assigned tasks

The workflow contains four language-model agents and one conditional-logic stage. The conditional stage is listed as Agent 5 in the project specification, but it is deterministic graph logic and does not make an LLM/API call.

Agent / stage	Input	Assigned task	Output and rules
Agent 1: Topic Extraction Agent	Full transcript and locally selected relevant transcript segments	Identify the distinct main subjects discussed. Prefer concise, specific topic labels; do not turn every sentence into a topic.	A JSON object containing `key_topics`, a list of topic strings.
Agent 2: Meeting Summary Agent	Full transcript and locally selected relevant transcript segments	Explain the purpose, principal discussion, and important outcomes of the meeting concisely.	A JSON object containing `meeting_summary`. The workflow validates that the summary contains 3–5 sentence-like units.
Agent 3: Action Item Extraction Agent	Full transcript and locally selected relevant transcript segments	Extract concrete tasks or commitments explicitly assigned or accepted during the meeting. Separate distinct tasks and identify the responsible person only when stated.	A JSON object containing `action_items`; each item has `task_description` and `owner`. Missing owners are normalized to `Owner: Not specified`. General discussion or suggestions without a commitment should not be presented as confirmed tasks.
Agent 4: Priority Classification Agent	Extracted action items and transcript context	Assign urgency to each task based on deadlines, urgency language, and the stated impact. Consider phrases such as “urgent,” “ASAP,” “by tomorrow,” “this week,” and “by Friday.”	A JSON object containing one priority for each action, selected from `High Priority`, `Medium Priority`, or `Low Priority`. The workflow checks that the number of priorities matches the number of actions.
Agent 5: Conditional Logic (no LLM)	Action list produced by Agent 3	Decide whether priority classification is needed.	If actions exist, route to Agent 4; if none exist, skip Agent 4 and route directly to finalization.
Final Output Node (no LLM)	Topic, summary, action, and (when applicable) priority results	Assemble a consistent user-facing report without generating additional analysis.	Return exactly `meeting_summary`, `key_topics`, and `action_items`. If there are no tasks, `action_items` is the exact message `No action items identified in this meeting.`

The agents are separate graph nodes, but they run sequentially rather than concurrently. Each AI agent receives the complete transcript as well as its query-relevant local excerpts. Agent 4 runs only on the action-bearing branch. The final output includes each task's owner and priority within the action item; it does not add separate top-level owner or meeting-priority fields.

4.4 Graph nodes

Node	Responsibility	Inputs and outputs
`input`	Validates and prepares the transcript	Rejects blank text; splits it into segments; computes local embeddings
`topics`	Finds main discussion themes	Returns a list of concise topic labels
`summary`	Summarizes the meeting	Returns a summary validated to 3–5 sentence-like units
`actions`	Extracts explicit commitments	Returns task descriptions and owner names; normalizes missing owners
`priority`	Classifies urgency for each task	Returns one priority label for each action item
`finalize`	Combines the completed results	Creates the stable report dictionary used by the app

The topic, summary, and action nodes are connected sequentially. After action extraction, `route_after_actions` checks whether the returned action list is truthy. If it is empty, the graph proceeds directly to finalization; otherwise it runs the priority-classification agent first. This conditional router is the workflow's fifth required component; it is logic, not a separate LLM. The finalizer returns only `meeting_summary`, `key_topics`, and `action_items`.

4.5 Agent prompting and structured responses

Each LLM stage has a separate instruction describing its task. The helper `invoke_structured` appends a plain Python dictionary describing the relevant JSON Schema to the instruction and calls `with_structured_output(None, method="json_mode")`. The decoded dictionary is then checked by the recursive `validate_json_value` function for expected JSON types, required keys, enumerated values, and unexpected keys. The notebook schemas describe topic lists, summaries, action items, and priorities. Additional functions validate the 3–5 sentence summary, normalize missing owners to `Owner: Not specified`, clean topic labels, and check that a priority is returned for every action. These checks are implemented with plain dictionaries and functions rather than Pydantic models or custom classes.

JSON mode requests a JSON-formatted response; it does not by itself guarantee that the model's answer is factually correct. Schema validation checks output shape and selected constraints, not truth against the transcript.

5. Models and AI techniques

5.1 Chat / generative model

Property	Configuration
Requested model identifier	`nvidia/nemotron-3-ultra-550b-a55b:free`
Provider gateway	OpenRouter
API base URL	`https://openrouter.ai/api/v1`
Client library	`langchain-openai` / `ChatOpenAI`
Generation temperature	`0`
Request timeout	90 seconds
Response format	JSON mode, with schema supplied in the prompt
Model execution location	Remote service reached through OpenRouter

The model identifier is the configured OpenRouter model slug. This application does not host or fine-tune the generative model. The model is used for topic extraction, summary generation, action extraction, and priority reasoning.

Actual provider-side model implementation, availability, throughput, and limits depend on OpenRouter and its upstream provider at the time of use.

5.2 Embedding model

Property	Configuration
Model identifier	`sentence-transformers/all-MiniLM-L6-v2`
Library integration	`langchain-huggingface` `HuggingFaceEmbeddings`
Device	CPU
Embedding normalization	Enabled
Execution location	Local machine after model download
Role	Rank transcript segments by semantic similarity to a query

The embedding model is separate from the chat model. It does not generate meeting notes. The notebook splits the input on blank/new lines and selected sentence-ending punctuation, embeds each segment once for the workflow run, embeds an agent-specific query, and ranks the segments by cosine similarity. The top eight segments (or all

available segments when fewer than eight exist) are added to the relevant stage's prompt. The embedding model object is memoized in-process with an LRU cache (maximum two model entries).

5.3 Retrieval design and its boundary

The locally selected excerpts provide additional, query-relevant evidence. However, the current implementation also includes the full transcript in each agent prompt. It is therefore more accurate to describe this as local semantic excerpt selection added to full-context prompting, rather than a strict retrieval-only RAG pipeline that replaces or truncates the transcript. The full transcript remains subject to the remote model's context limit.

5.4 No model training or fine-tuning

This is an inference-time application. No training dataset, fine-tuning run, weight update, adapter, or custom model checkpoint is included in the project. Dictionary schemas, validation functions, instructions, graph nodes, and priority guidance configure the inference workflow; they do not constitute model training.

6. Priority classification policy

The priority node is instructed to choose one of three labels for every action:

- **High Priority:** explicit urgent/ASAP language, same-day deadlines, critical customer or service impact, or deadlines by tomorrow.
- **Medium Priority:** near-term deadlines such as this week or Friday, or clear commitments without the strongest urgency signals.
- **Low Priority:** no stated urgency or deadline.

Classification is performed by the generative model using the transcript and actions. These descriptions are prompting guidance, not a deterministic rules engine, and the system does not independently calculate a priority score. The output contains a priority for each task and does not calculate or display an overall meeting priority. The workflow checks that the priority agent returns the same number of task priorities as the number of extracted action items. If this check fails, it raises an error instead of silently inventing or omitting priority results.

7. Technology stack

7.1 Direct project dependencies

The following versions were present in the project's Python 3.14.6 virtual environment when this report was prepared. `requirements.txt` specifies lower bounds rather than exact pins, so a fresh installation may resolve to newer compatible versions.

Package	Installed version	Purpose
Python	3.14.6	Runtime
`langgraph`	1.2.14	Stateful directed workflow and conditional routing
`langchain-openai`	1.7.0	OpenAI-compatible client for OpenRouter
`langchain-huggingface`	1.2.3	Hugging Face embedding integration
`sentence-transformers`	6.1.0	Local sentence embedding model
`streamlit`	1.65.0	Interactive web UI
`python-dotenv`	1.2.4	Load local `.env` configuration
`ipykernel`	7.4.0	Jupyter kernel integration
`jupyter`	1.1.1	Notebook environment

8. User interface and application behavior

8.1 Banner and layout

The Streamlit page uses a wide layout and an HTML/CSS banner rendered with `st.html`. The banner has a dark blue-to-teal gradient, an accent glow, concentric decorative circles, a project heading, a brief explanatory sentence, and feature labels. The banner content is static application markup; the user transcript is not inserted into it.

8.2 Transcript input

The main page contains a labelled text area. If `Meeting_Transcript.txt` exists, its contents are loaded as the initial value. Users can replace or edit that text. The app validates that the transcript is not blank before calling the graph.

8.3 Results

After successful analysis, the app displays exactly these three report sections:

Meeting Summary — concise summary.
Key Topics — numbered topic list.
Action Items — task, owner, and priority table, or the exact no-action-items message.

8.4 API-key configuration

The API key is obtained from Streamlit secrets or the `OPENROUTER_API_KEY` environment variable, with a local `.env` file loaded by `python-dotenv`.

The `.env` file is excluded by `.gitignore` and must not be committed.

9. Setup and operation

9.1 Configure the API key

Use one of these mechanisms:

- Set `OPENROUTER_API_KEY` in the process or deployment environment.
- Put `OPENROUTER_API_KEY=...` in a local `.env` file.
- Configure `OPENROUTER_API_KEY` in Streamlit secrets.

Do not put a real key in source code, notebook output, a screenshot, or a version-controlled file.

The Streamlit UI intentionally provides no model or key controls.

9.3 Run the notebook

Open `MeetingTranscriptAnalyzer.ipynb` with the project virtual environment as the kernel. Run the workflow-definition code cell, then run the analysis cell. The analysis cell reads `Meeting_Transcript.txt` relative to the current working directory. Set the API key before running it.

9.4 Run the web application

```
streamlit run app.py
```

The app begins with the sample transcript loaded. Replace it with another transcript and select **Analyze meeting** to run the workflow.

10. Error handling and operational behavior

- Blank transcripts are rejected by both the interface and graph input node.
- Missing API-key configuration is reported in the UI before graph execution.
- Model, provider, parsing, and graph exceptions are surfaced as an error message and exception detail in the Streamlit app; they are not converted into successful-looking empty results.
- Invalid structured output is rejected through dictionary-schema validation functions.
- A mismatch between extracted actions and returned task priorities raises an explicit error.
- The first local embedding-model initialization may take longer because it loads or downloads model files.
- LLM network requests use a 90-second timeout in the notebook workflow. Exception details shown in the browser can expose internal diagnostic information. For a public deployment, replace detailed tracebacks with user-safe messages and write diagnostics to access-controlled server logs.

11. Verification and evaluation status

11.1 Verification performed

The following implementation checks have been performed during development:

- Installed dependency consistency check (`pip check`) completed with no broken requirements.
- Notebook JSON and its Python code cells were parsed for syntax.
- The app's notebook loader was exercised with the renamed notebook.
- Deterministic mock-model tests exercised the action path and confirmed that priority classification runs when actions are present.
- A deterministic mock-model test confirmed that priority classification is skipped when the action list is empty.
- Blank-transcript handling was tested at the graph input.

11.2 Not yet evaluated

The checks above establish application wiring and selected control flow, not the quality of the model's analysis. No statistically meaningful accuracy, precision, recall, F1 score, latency benchmark, user study, or comparison against human minutes has been measured. Mock outputs do not test OpenRouter's live response behavior. A live end-to-end run must be performed with a valid key and available provider before claiming that the deployed model succeeds for a particular transcript.

11.3 Recommended evaluation plan

Before making quality claims:

Collect a diverse set of transcripts for which data use is authorized. Ask at least one human reviewer (preferably two for a subset) to annotate topics, a reference summary, explicit action items, owners, and urgency. Define annotation guidelines for implicit owners, duplicate actions, tentative language, decisions, and ambiguous deadlines.
Evaluate each component separately:

- Topic extraction: human relevance/coverage ratings or a documented topic-label matching method.
- Summary: factual-consistency, coverage, and readability ratings from human reviewers; automatic overlap metrics alone are insufficient.
- Action items: precision, recall, and F1 using a clearly defined match rule for task descriptions.
- Owners: exact/normalized owner accuracy, with a separate score for correctly identifying unspecified owners.
- Priority: per-task accuracy, confusion matrix, and class-wise precision/recall/F1.

Record model identifier, date, prompt version, transcript length, and failures for reproducibility; never log secrets.

Include edge cases such as no tasks, multiple owners, tentative suggestions, conflicting deadlines, multilingual text, and long transcripts. Report sample size, annotation procedure, metric definitions, and limitations alongside results.

12. Limitations

12.1 Model and reasoning limitations

- Generative models can omit, merge, misstate, or invent details, including tasks, names, deadlines, and priorities.
- Dictionary-schema validation checks structure and selected field constraints; it does not prove that a claim is supported by the transcript.
- Priority labels follow prompt guidance, not an auditable deterministic scoring system. Similar transcripts can receive different labels across model versions or provider behavior.
- A task owner may be ambiguous, referred to by a role, or mentioned in a different statement. The current implementation does not resolve identity against a company directory.
- The 3–5 sentence validator counts sentence-like spans by punctuation and whitespace. Abbreviations, decimal numbers, or unusual punctuation can cause it to count sentences imperfectly.

12.2 Input and context limitations

- Only text input is supported. Audio transcription, video, file-upload parsing, and automatic speaker diarization are not implemented.
- Segmentation uses simple line and punctuation splitting; it does not preserve a formal transcript structure or perform discourse-aware chunking.
- The full transcript is sent to each relevant LLM call together with retrieved excerpts. Very long transcripts may exceed upstream context limits, raise request errors, or incur more latency and cost.
- There is no explicit maximum transcript length, truncation strategy, token counter, or map-reduce summary strategy.
- The local excerpt ranking uses one generic query per stage and a maximum of eight segments. Ranking may miss relevant context, especially where the meeting uses domain-specific terms or pronouns.

12.3 Provider and deployment limitations

- LLM analysis requires an internet connection, valid OpenRouter credentials, and provider/model availability.
- A `:free` model endpoint may have changed availability, quotas, throughput, or provider-side behavior; no service-level guarantee is implemented here.
- The project does not retry rate limits, fail over to another model, queue jobs, or stream partial model results.
- Embedding computation is local but CPU-based. First-use model download and memory use depend on the host machine and cache state.
- This prototype has no user authentication, role-based access control, multi-tenant isolation design, audit log, persistent report store, or administrative controls.
- The application currently displays exception details in the UI and should be hardened before public or production deployment.
- Dependency constraints use minimum versions, not a lock file. Recreating the environment later may install different versions.

12.4 Security and privacy limitations

- Hiding the key in the UI does not make the transcript local: the full transcript is sent to OpenRouter for language-model inference.
- A local `.env` file is excluded from Git by the current .gitignore, but its filesystem permissions and backups remain the operator's responsibility.
- The workflow executes source code from the project notebook's tagged runtime cell. Only run the app with a trusted, reviewed notebook.
- Prompt injections or adversarial instructions embedded in a transcript may influence model output; the current prompts do not provide a specialized prompt-injection defense or isolation layer.
- No formal security assessment, privacy impact assessment, or penetration test is included.

13. Conclusion

The project demonstrates an end-to-end pattern for turning a meeting transcript into structured notes with a graph-orchestrated LLM workflow. It combines separate extraction stages, dictionary-schema-validated JSON output, local semantic segment ranking, conditional routing, and a browser interface. Its most important design distinction is that embeddings run locally while language-model analysis uses a remote OpenRouter endpoint. The implementation is suitable as a capstone prototype and a foundation for further experimentation. It should not be described as production-grade or quantitatively validated until the recommended human-grounded evaluation, privacy review, and deployment hardening have been completed.

Appendix A: Priority labels and output fields

Output area	Field	Meaning
Summary	`meeting_summary`	Concise account of the meeting
Topics	`key_topics`	Distinct high-level subjects discussed
Actions	`task_description`	Concrete follow-up commitment
Ownership	`owner`	Named person or `Owner: Not specified`
Task urgency	`priority`	High, Medium, or Low Priority for one action
Empty action list	`action_items`	`No action items identified in this meeting.`